claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-ad7916fab73825244.jsonl`
- SHA-256: `206d8ddad4abe0c1c166fde030488d1db7a8f20a33db7507ab46d4090c215b4a`
- Source modified: `2026-07-07T15:18:45+00:00`
- Imported at: `2026-07-09T08:32:24+00:00`
- project: `wf_a8f43370-4cb`
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`

Transcript

1. user (2026-07-07T15:13:07.032Z)

```
You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run
control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process
segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job
GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic
"WASB weights not found at /opt/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):
- BACKEND (repo undefined, branch undefined, PR #undefined):
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,
  fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote
the
  choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-
backed
  AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-
process as
  before). Plus 4 tests in tests/test_api_cloud_dispatch.py.
- SPA (repo undefined, branch undefined, PR #undefined):
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when
  capabilities.deployment.compute_target==="cloud_run" (else keeps the free "local" default).
Plus 1 test.

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:
- undefined/backend.diff                    (backend commit diff)
- undefined/orchestration_new.py            (FULL post-change backend/orchestration.py)
- undefined/test_api_cloud_dispatch_new.py  (FULL post-change tests)
- undefined/spa.diff                        (SPA commit diff)
- undefined/IngestPanel_new.tsx             (FULL post-change IngestPanel.tsx)
You MAY read other repo files for ground truth, e.g. under undefined:
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);
and under undefined: web/src/api/types.ts, web/src/api/client.ts,
web/src/components/IngestPanel.test.tsx.
You may also run: gh pr diff undefined -R Khelsutra/badminton-highlight-indexer  and  gh pr diff
undefined -R avidullu/khelsutra.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a
regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete
config/inputs
```

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.
If nothing real exists in your dimension, return an empty findings array.

Adversarially VERIFY this finding from the 'test-adequacy' review. Your job is to REFUTE it if at all possible: read the ACTUAL authoritative code (the snapshot files above and any repo files needed) and determine whether the described failure REALLY occurs against the real code as written. Default to REFUTED unless you can reproduce it concretely. Beware findings that assume a code path that does not exist, or ignore an earlier guard.

FINDING:
- severity: low
- category: test-coverage
- location: tests/test_api_cloud_dispatch.py::test_local_weights_backed_on_pure_local_server_is_not_guardrailed (snapshot lines ~801-829)
- claim: The guardrail fires only when _cloud_available(cfg) is True, which has two routes: compute_target=='cloud_run' (config) and CLOUD_RUN_JOB+GOOGLE_CLOUD_PROJECT+storage.output_root (env). Every guardrail-FIRES test uses the config route (cloud_env sets compute_target=cloud_run); the env route into the guardrail is never exercised ? the 'not guardrailed' test explicitly deletes both env vars AND uses compute_target='', so compute_target=''-but-cloud-wired-via-env is untested.
- failure_scenario: A local-default box wired for cloud via env (deployment.compute_target='', CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT + storage.output_root set) is cloud-available through _cloud_available's env branch (orchestration.py:202-206). A compute='local' + wasb_hybrid request with no local weights should fail-fast there too; if that env branch of the guardrail's fire condition regressed, the box would silently fall through to the deep 'weights not found' failure and no guardrail test would catch it. Mitigant: _cloud_available's env branch is independently covered by test_per_request_cloud_forces_dispatch_on_local_server / test_cloud_override_survives_api_boundary_on_local_server, so real-world risk is low ? this is a completeness gap, not a likely latent bug.
- suggested_fix: Add one guardrail test on a compute_target='' server that sets CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT + a storage.output_root bucket, segmenter='wasb_hybrid', compute='local', no local weights, and asserts the fast-fail (status=failed, message 'can't run detection locally', fake.calls==[], path='not_run'), proving the env-wired cloud_available route also triggers the guardrail.

Return CONFIRMED only with a concrete repro grounded in the real code; else REFUTED with the reason.

2. assistant (2026-07-07T15:13:14.518Z)

I'll start by locating the snapshot files and the relevant code. The "undefined" placeholders suggest I need to find the actual paths.

3. user (2026-07-07T15:13:16.783Z)

```
total 62
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:38 .git
-rw-r--r-- 1 avidu 197609  297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609  323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web
---PARENT---
total 108
```

```
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609  905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
```

4. user (2026-07-07T15:15:18.783Z)

```
Command running in background with ID: b0jyw09pn. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\tasks\b0jyw09pn.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

5. assistant (2026-07-07T15:15:24.698Z)

```
The real backend code is in `badminton-highlight-indexer`. Let me explore that and check for
snapshot files.
```

6. user (2026-07-07T15:15:27.698Z)

```
total 650
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:43 .
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609     0 Jun 25 23:31 .claude
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609  1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:44 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609     0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609  1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609     0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609     0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609     0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609     0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609     0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609     0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609  6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609  2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609     0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:43 docs
drwxr-xr-x 1 avidu 197609     0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609  1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:14 output
-rw-r--r-- 1 avidu 197609  5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609  1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609   585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609   809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609  2118 Jun 29 13:47 run_all_tests.py
```

```
-rw-r--r-- 1 avidu 197609  5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609  4594 Jun 29 13:47 run_regression.py
-rwxr-xr-x 1 avidu 197609  5073 Jul  7 10:27 score_golden_gate.py
-rwxr-xr-x 1 avidu 197609  7617 Jul  7 03:32 score_local.py
drwxr-xr-x 1 avidu 197609     0 Jul  7 08:50 scratch
drwxr-xr-x 1 avidu 197609     0 Jul  5 19:39 scripts
-rw-r--r-- 1 avidu 197609  2410 Jul  4 06:58 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:43 tests
drwxr-xr-x 1 avidu 197609     0 Jul  2 19:37 tools
drwxr-xr-x 1 avidu 197609     0 Jun 10 15:57 training
---BHI-WT-WORKER-REVAL---
total 362
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:34 .
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 ..
-rw-r--r-- 2 avidu 197609 94208 Jul  7 20:34 .coverage
-rw-r--r-- 1 avidu 197609   611 Jul  7 19:57 .coveragerc
-rw-r--r-- 1 avidu 197609  1059 Jul  7 19:57 .gcloudignore
-rw-r--r-- 1 avidu 197609   110 Jul  7 19:57 .git
-rw-r--r-- 1 avidu 197609   947 Jul  7 19:57 .gitattributes
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 .github
-rw-r--r-- 1 avidu 197609  1998 Jul  7 19:57 .gitignore
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:08 .mypy_cache
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:06 .pytest_cache
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:06 .ruff_cache
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 .vscode
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:09 __pycache__
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:05 backend
-rw-r--r-- 1 avidu 197609  6619 Jul  7 19:57 CLAUDE.md
-rw-r--r-- 1 avidu 197609  2735 Jul  7 19:57 config.json
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 deploy
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 docs
drwxr-xr-x 1 avidu 197609     0 Jul  7 19:57 eval_baselines
-rw-r--r-- 1 avidu 197609  1999 Jul  7 19:57 mypy.ini
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:34 output
-rw-r--r-- 1 avidu 197609  5413 Jul  7 19:57 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jul  7 19:57 pytest.ini
-rw-r--r-- 1 avidu 197609 35511 Jul  7 19:57 README.md
-rw-r--r-- 1 avidu 197609  1674 Jul  7 19:57 requirements.txt
-rw-r--r-- 1 avidu 197609   585 Jul  7 19:57 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609   809 Jul  7 19:57 ruff.toml
-rwxr-xr-x 1 avidu 197609  2118 Jul  7 19:57 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5739 Jul  7 19:57 run_eval.py
-rw-r--r-- 1 avidu 197609  4594 Jul  7 19:57 run_regression.py
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:09 scripts
-rw-r--r-- 1 avidu 197609  2410 Jul  7 19:57 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:28 tests
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:09 tools
drwxr-xr-x 1 avidu 197609     0 Jul  7 20:09 training
```

7. user (2026-07-07T15:15:29.517Z)

```
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_new.py
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr35.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr37.diff
```

C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr38.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr39.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr50.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr51.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr52.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr53.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr54.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr55.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr57.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/pr58.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/pr55.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-indexer/edc346c2-92fc-4ad1-b556-c7ab955a5613/scratchpad/pr158.diff

8. assistant (2026-07-07T15:15:40.841Z)

Found the snapshot files. Let me read the authoritative orchestration and test files.

9. user (2026-07-07T15:15:42.303Z)

```
1       """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3       The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4       Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5       and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6       Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8       SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9       monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10      ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11      the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12      bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13      ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14      This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15      Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16      ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17      Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18      objects and need no routing.
```

```
    """

    import csv
    import json
    import logging
    import math
    import os
    import re
    import tempfile
    import threading
    from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
    from urllib.parse import quote

    from backend import storage
    from backend.config import AppConfig, StitchingConfig
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
    from backend.results import Failure
    from backend.utils.paths import output_base, safe_output_filename

    if (
        TYPE_CHECKING
    ):  # runtime import would be circular (main imports us for the re-export)
        from backend.main import ProcessRequest  # noqa: F401

    logger = logging.getLogger(__name__)

    # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
    # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
    # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
    # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
    # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
    # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
    _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)


    def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    def _validation_failure_message(failures: List[Any]) -> str:
        details = []
        for failure in failures:
            name = getattr(failure, "validator_name", None) or "validation"
            message = (getattr(failure, "message", None) or "").strip()
            details.append(f"{name}: {message}" if message else str(name))
        if not details:
            return "Video failed validation checks."
```

```
        return "Video failed validation: " + "; ".join(details)


    def _report_config_for_input(config: Any, raw_input: str) -> Any:
        """Return report config that keeps remote-input reports out of disposable
scratch."""
        storage_config = (
            config.get("storage")
            if isinstance(config, dict)
            else getattr(config, "storage", None)
        )
        try:
            if storage.input_is_local(raw_input, storage_config):
                return config
        except ValueError:
            return config
        try:
            cfg = (
                config.model_dump(mode="json")
                if hasattr(config, "model_dump")
                else dict(config or {})
            )
        except Exception:
            cfg = {}
        report = dict(cfg.get("report") or {})
        if report.get("output_dir"):
            return config
        report["output_dir"] = os.path.join(output_base(config), "reports")
        cfg["report"] = report
        return cfg


    def _ingest_remote_segments(
        db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
    ) -> int:
        """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
        persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
        ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
        Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
        backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
        skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
        cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
        and prunes any partial rows (destroy-AFTER-confirm)."""
        uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
        ref = storage.parse_uri(uri)
        n = 0
        with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
            )
            with open(local, newline="", encoding="utf-8") as f:
                for row in csv.DictReader(f):
                    if n >= max_rows:
                        logger.warning(
                            "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
                            max_rows,
                            uri,
```

```
                        )
                        break
                    try:
                        start, end = float(row["start"]), float(row["end"])
                    except (KeyError, TypeError, ValueError):
                        continue
                    if not (math.isfinite(start) and math.isfinite(end)):
                        continue  # reject inf/nan times rather than persist a junk rally
                    meta: Dict[str, Any] = {
                        "source": "cloud_run",
                        "ending_reason": (row.get("ending_reason") or "").strip(),
                    }
                    sc = (row.get("shot_count") or "").strip()
                    if sc:
                        try:
                            meta["shot_count"] = int(
                                float(sc)
                            )  # OverflowError on 'inf'/'1e400'
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
    # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
    # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
    # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
    # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
    _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})


    def _local_detector_weights_present(cfg, seg_name: str) -> bool:
        """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
        on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
        and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
        otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
        A non-weights segmenter returns True (nothing local to check)."""
        name = (seg_name or "").strip().lower()
        if name not in _WEIGHTS_BACKED_SEGMENTERS:
            return True
        idx = getattr(cfg, "indexing", None)
        substrate = (
            getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
        ).strip().lower()
        if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
            path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
        else:  # wasb_hybrid, or fusion on the (default) WASB substrate
            path = (
                os.environ.get("WASB_WEIGHTS")
                or getattr(getattr(idx, "wasb", None), "weights_path", "")
                or ""
            )
```

```
        return bool(path) and os.path.exists(os.path.expanduser(path))


    def _cloud_available(cfg) -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.

        Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
        deployment = getattr(cfg, "deployment", None)
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(
            os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
        )
        out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        return bool(has_job and out_root)


    def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
        """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
        minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
        with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
        (queue/cold-start), so the very first tick already shows movement."""
        minutes = int(elapsed_sec) // 60
        if minutes < 1:
            return f"segmenting ({seg_name}) on cloud GPU ? starting up"
        return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"


    def submit_time_video_id(input_ref, cfg) -> Optional[str]:
        """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
        object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
        deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
        ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
        if getattr(input_ref, "backend", "") != "gcs":
            return None
        sc = _storage_settings(cfg)
        if not sc:
            return None
        try:
            st = storage.get_storage("gcs", config=sc).stat(input_ref)
            return getattr(st, "md5", None) or None
        except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
            logger.info(
                "[API] submit-time md5 unavailable for %s (metadata stat failed)",
                getattr(input_ref, "uri", input_ref),
                exc_info=True,
            )
            return None

```

```
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
for
246         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247         return {
248             "video_id": video_id,
249             "status": "success",
250             "message": (
251                 f"Reused {len(segments)} cached play segments. "
252                 "Pass force=true to reprocess this video."
253             ),
254             "results": validation_results or [],
255             "play_segments": segments,
256             "cached": True,
257             "compute": {"path": "cached", "requested": requested_compute},
258         }
259
260
261     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264         ref = storage.parse_uri(uri)
265         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
266             local = storage.fetch(
267                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
268             )
269             with open(local, encoding="utf-8") as f:
270                 loaded = json.load(f)
271         return loaded if isinstance(loaded, dict) else None
272
273
274     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
275     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
276     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
277     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
278     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
279     _REHYDRATE_LOCKS_GUARD = threading.Lock()
280
281
282     def _rehydrate_lock(video_id: str) -> threading.Lock:
283         with _REHYDRATE_LOCKS_GUARD:
284             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287     def probe_process_cache(db, cfg, video_id: str) -> bool:
288         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293         errors report False (? the normal dispatch path with its guards)."""
294         try:
295             existing = db.get_video(video_id)
296             if (
297                 existing
```

```
                and existing.get("status") == "processed"
                and db.query_play_segments(video_id, {})
            ):
                return True
            sc = _storage_settings(cfg)
            if not sc or (sc.get("backend") or "local") == "local":
                return False
            output_root = (sc.get("output_root") or "").rstrip("/")
            if not output_root:
                return False
            report_uri = f"{output_root}/outputs/{video_id}/report.json"
            if not storage.exists(report_uri, config=sc):
                return False
            report = _read_bucket_json(report_uri, sc)
            return bool(report and report.get("status") == "success")
        except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
            logger.warning(
                "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
            )
            return False


    def cached_process_result(
        db, cfg, video_id: str, video_path: str, requested_compute=None
    ) -> Optional[Dict[str, Any]]:
        """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
        the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
        re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
        source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
        completed work found (or the check failed) ? the caller dispatches normally. The
bucket
        branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
        partial earlier ingest is replaced, never double-counted.

        Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
        per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
        in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
        job before invoking (writes happen post-claim)."""
        try:
            with _rehydrate_lock(video_id):
                existing = db.get_video(video_id)
                if existing and existing.get("status") == "processed":
                    segs = db.query_play_segments(video_id, {})
                    if segs:
                        return _cached_process_payload(
                            video_id,
                            existing.get("validation_results", []),
                            segs,
                            requested_compute,
                        )
                sc = _storage_settings(cfg)
                if not sc or (sc.get("backend") or "local") == "local":
                    return None
                output_root = (sc.get("output_root") or "").rstrip("/")
                if not output_root:
                    return None
                outputs_uri = f"{output_root}/outputs/{video_id}"
                report_uri = f"{outputs_uri}/report.json"
                if not storage.exists(report_uri, config=sc):
```

```
                    return None
                report = _read_bucket_json(report_uri, sc)
                if not report or report.get("status") != "success":
                    return None
                if not video_path:
                    # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
                    # carries the worker's --video-uri (P3); older reports fall back to the
                    # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
                    # recover, and the next same-md5 submit re-binds the truthful path via
                    # add_video's upsert.
                    video_path = str(report.get("video_uri") or "") or _first_object_under(
                        f"{output_root}/videos/{video_id}/", sc
                    )
                play_wm, cand_wm = db.segment_watermarks(video_id)
                db.add_video(video_id, video_path, "processed", [])
                n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
                if n <= 0:
                    return None
                segments = [
                    s
                    for s in db.query_play_segments(video_id, {})
                    if int(s.get("id", 0)) > play_wm
                ]
                _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
                if not segments:
                    return None
                logger.info(
                    "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
                    n,
                    video_id,
                    outputs_uri,
                )
                return _cached_process_payload(video_id, [], segments, requested_compute)
        except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
            logger.warning(
                "[API] submit-time cache check failed for %s ? dispatching normally",
                video_id,
                exc_info=True,
            )
            return None


    def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
        """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
        try:
            return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
        except Exception:  # noqa: BLE001 ? recovery nicety, never a failure
            return ""


    _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")


    def read_path_rehydrate(db, cfg, video_id: str) -> bool:
        """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
        nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
        ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
        has the video's rows now (re-query). Non-md5 ids return False without touching the
        bucket; every failure degrades to False (the read just reports what the DB has)."""
        vid = (video_id or "").strip().lower()
```

```
        if not _MD5_HEX_RE.fullmatch(vid):
            return False
        if not probe_process_cache(db, cfg, vid):
            return False
        return cached_process_result(db, cfg, vid, "") is not None


    def _maybe_dispatch_remote(
        cfg,
        db,
        req: "ProcessRequest",
        video_id: str,
        seg_name: str,
        val_results,
        play_wm: int,
        cand_wm: int,
        notify,
    ) -> Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
            # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
            # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
            # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
            return (
                {
                    "video_id": video_id,
                    "status": "failed",
                    "message": msg,
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                    "compute": {
                        "path": "cloud_run" if ran else "not_run",
                        "requested": req.compute,
                        "target": "cloud_run",
                        "dispatched": ran,
                    },
                },
                ran,
            )

```

```
        # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
        # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
        # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
        choice = (req.compute or "").strip().lower()
        if choice and choice not in ("local", "cloud"):
            logger.warning(
                "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
                req.compute,
            )
            choice = ""
        if choice == "local":
            # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
            # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
            # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
            # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
            # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
            # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
            # below exactly as before (the weights-present check is the escape hatch).
            if (
                (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
                and _cloud_available(cfg)
                and not _local_detector_weights_present(cfg, seg_name)
            ):
                return _failed(
                    f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
                    "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
                    False,
                )
            return None  # explicit "run on my machine" ? in-process regardless of the
server's target
        if choice == "cloud":
            if not _cloud_available(cfg):
                return _failed(
                    "cloud-serving was requested but this server isn't configured for it "
                    "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
                    False,
                )
            compute = get_compute_backend(cfg, target_override="cloud_run")
        else:
            compute = get_compute_backend(
                cfg
            )  # server default (default-OFF unless cloud_run)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(cfg, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
```

```
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
            # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
            # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
            # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
            # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
            # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
            # default, while a bare/local-looking root can't stage no matter what
input_backend says
            # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
            # gives us nothing to build the staged URI under).
            input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
            try:
                root_backend = (
                    storage.parse_uri(input_root).backend if input_root else "local"
                )
            except ValueError:  # unsupported scheme in input_root ? nowhere real to stage
to
                root_backend = "local"
            if root_backend == "local":
                return _failed(
                    "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
                    "stage local uploads into; a path local to this machine can't be read by
the Cloud "
                    "Run job.",
                    False,
                )
            local_src = input_ref.key  # resolved local path of the upload
            staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
            try:
                notify("staging upload to bucket")
                # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
                # region/addressing for R2·DO·MinIO; GCS project/client) ? not a fresh
default client.
                storage.put(
                    local_src,
                    staged_uri,
                    config=(
                        storage_cfg.model_dump() if storage_cfg is not None else None
                    ),
                )
            except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
                logger.error(
                    "[API] cloud-serving: failed to stage %s -> %s: %s",
                    local_src,
                    staged_uri,
                    e,
                    exc_info=True,
                )
                return _failed(
                    f"cloud-serving: failed to stage the uploaded file to {input_root}:
```

```
{e}",
                    False,
                )
            # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
            # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
            # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
            req.video_path = staged_uri
            input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
            db.add_video(
                video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
            )
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed(
                "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
                "? that's where the Cloud Run job writes the result.",
                False,
            )

        notify("dispatching (cloud_run)")
        logger.info(
            "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
            req.video_path,
            out_root,
        )
        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        idx_cfg = getattr(cfg, "indexing", None)
        sk = getattr(idx_cfg, "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
        # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
        # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
        # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
        # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
        # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
        wasb_cfg = getattr(idx_cfg, "wasb", None)
        if wasb_cfg is not None:
            decode_backend = getattr(wasb_cfg, "decode_backend", None)
            if decode_backend:
                overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
            batch_size = getattr(wasb_cfg, "batch_size", None)
            if batch_size:
                overrides.append(f"indexing.wasb.batch_size={batch_size}")
            prefetch = getattr(wasb_cfg, "prefetch_batches", None)
            if prefetch:
                overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
```

```
        spec = ComputeJobSpec(
            video_uri=req.video_path,
            out_uri=out_root,
            segmenter=seg_name,
            config_overrides=tuple(overrides),
        )
        try:
            # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
            # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
            # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
            # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
            result = compute.run_infer(
                spec,
                on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning(
                "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
                "and the worker hash; ingesting under the local id.",
                result.video_id,
                video_id,
            )

        notify("ingesting (cloud_run)")
        storage_dict = (
            storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
            if hasattr(storage_cfg, "model_dump")
            else {}
        )
        try:
            _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(
                f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
                True,
            )

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [
            s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
        ]
        _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
```

```
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info(
            "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
            result.execution,
            provenance["job"],
            provenance["region"],
            result.outputs_uri,
            video_id,
        )
        return (
            {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )


    def _execute_processing(
        config=None, db=None, req=None, progress=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        import backend.main as _main  # call-time seam: main's globals/model stay patchable

        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
```

```
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = ""
        try:
            local_video = storage.acquire_local_input(
                req.video_path, getattr(config, "storage", None)
            )
            video_id = _main.compute_video_id(local_video)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        try:
            existing_video = db.get_video(video_id)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        was_reingest = existing_video is not None

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(
            getattr(config, "indexing", None), "default_segmenter", "motion"
        )
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        val_results: List[Any] = []
        val_context: Dict[str, Any] = {}
        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
        compute_actual: Optional[str] = None
        response: Optional[Dict[str, Any]] = None
        try:
            if (
                existing_video
                and existing_video.get("status") == "processed"
                and not req.force
            ):
                cached_segments = db.query_play_segments(video_id, {})
                if cached_segments:
                    logger.info(
                        "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
                        video_id,
                    )
                    response = _cached_process_payload(
                        video_id,
                        existing_video.get("validation_results", []),
                        cached_segments,
                        getattr(req, "compute", None),
                    )
                    return response

```

```
            # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
            notify("validating")
            logger.info(f"Running validations for {req.video_path}...")
            val_results, val_context = registry.run_all_with_context(local_video, config)
            failures = [r for r in val_results if not r.passed]

            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
                )
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": _validation_failure_message(failures),
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db.add_video(
                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
            )

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(
                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
```

```
            )
            if remote is not None:
                response, segmentation_ran = (
                    remote  # `ran` = did the cloud job actually execute (report)
                )
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db, config)
            try:
                # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
                # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
                with _LOCAL_COMPUTE_LANE:
                    result = segmenter.process_video(
                        video_id, local_video, req.player_pool, req.max_frames
                    )
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
```

```
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = _main.emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=_report_config_for_input(config, req.video_path),
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )


    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
        to a no-op rather than blocking a job it cannot measure."""
        try:
            sc = (
                storage_config.model_dump()
                if hasattr(storage_config, "model_dump")
                else (storage_config or {})
            )
            backend = storage.get_storage(input_ref.backend, config=sc)
            return int(backend.stat(input_ref).size)
        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
```

```
not block
            logger.debug(
                "free-space: could not stat remote source %s: %s", input_ref.uri, e
            )
            return None


    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""

        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
        row could change between submit and run). Verifies the video exists, the requested
segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
```

```
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
                f"stitching.min_shot_count={stitching.min_shot_count}.",
            )
        return video, kept, out_name


    def reel_object_uri(cfg, video_id: str, name: str) -> str:
        """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
        the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
        mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
        root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        if not root or "://" not in root:
            return ""
        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"


    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
        isn't silently constructed as a fresh default."""
        sc = getattr(cfg, "storage", None)
        return sc.model_dump() if sc is not None else None


    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
        reel_uri = reel_object_uri(cfg, video_id, name)
        if not reel_uri:
            return None
        settings = _storage_settings(cfg)
        if storage.exists(reel_uri, config=settings):
            return storage.signed_url(reel_uri, config=settings, method="GET")
        return None


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
```

```
        locale = params.get("locale")

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        local_src = ""
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                video["filepath"], getattr(cfg, "storage", None)
            )
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
        finally:
            if local_src:
                storage.cleanup_acquired_local_input(
                    video["filepath"], local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
```

```
(the endpoint
1125        # still falls back to the local file on the same instance).
1126        reel_uri = reel_object_uri(cfg, video_id, out_name)
1127        if reel_uri:
1128            try:
1129                notify("uploading reel")
1130                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1131                # project/client) is used, not a fresh default.
1132                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1133            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1134                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1135
1136        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1137        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1138        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1139        notify("finished")
1140        return {
1141            "status": "success",
1142            "filepath": out_path,
1143            "download_url": download_url,
1144            "video_id": video_id,
1145        }
1146
```

10. user (2026-07-07T15:15:42.826Z)

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
9
10      import csv
11      import json
12      import os
13
14      import pytest
15
16      pytest.importorskip("httpx")
17      from fastapi.testclient import TestClient
18
19      import backend.compute as compute_pkg
20      import backend.main as m
21      from backend.compute.cloud_run import CloudRunJobClient
22      from backend.config.models import AppConfig
23      from backend.infrastructure.database import Database
24      from backend.infrastructure.execution_policy import ExecutionPolicy
25      from backend.pipeline.validators.base import ValidationResult
26
27      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30      def _pass():
31          return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
```

```

    class _InlineExecutor:
        def submit(self, fn, *a, **k):
            fn(*a, **k)


    class _FakeCloudClient(CloudRunJobClient):
        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = (
                rows
                if rows is not None
                else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
            )

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump(
                        {
                            "video_id": self.video_id,
                            "segment_count": len(self.rows),
                            "status": "success",
                        },
                        f,
                    )
                with open(
                    os.path.join(outputs, "intervals.csv"),
                    "w",
                    newline="",
                    encoding="utf-8",
                ) as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "returncode": self.returncode,
                        "segment_count": len(self.rows),
                        "status": "success",
                    },
                    f,
                )
            return "exec-1"

        def cancel_execution(self, execution):
            raise AssertionError(
                "cancel must never fire on the happy path"
            )  # pragma: no cover

```

```

    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for hash+validate."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": "cloud_run"},
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token. ``**kw``
        forwards the per-request ``target_override`` (item 3) so the cloud-picker path resolves too."""
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(
                config, run_client=fake, policy=_FAST, token="tok", **kw
            ),
        )


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

```

```
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


    def test_cloud_dispatch_builds_correct_spec(cloud_env):
        _db, bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )

        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
        assert argv[argv.index("--out-uri") + 1] == str(bucket)
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        joined = " ".join(argv)
        assert (
            "deployment.compute_target=local_gpu" in joined
        )  # container runs INLINE (never re-dispatch)
        assert "indexing.detector_impl=native" in joined
        assert (
            "indexing.skip_ai_handoff=false" in joined
        )  # the engine's AI-handoff stance is forwarded


    def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
        """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
        Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.skip_ai_handoff = True
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])


    def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
        """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
preset ?
        the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
GPU-feed
        tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
making the
        preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
wiring."""
        _db, _bucket, monkeypatch = cloud_env
        m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
        m.global_config.indexing.wasb.batch_size = 64
        m.global_config.indexing.wasb.prefetch_batches = 4
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
```

```
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=nvdec_resident" in joined
        assert "indexing.wasb.batch_size=64" in joined
        assert "indexing.wasb.prefetch_batches=4" in joined


    def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
        """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
control-plane),
        but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
the worker's
        parity default; forwarding an empty value would reach argparse as a bad token)."""
        _db, _bucket, monkeypatch = cloud_env
        # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        joined = " ".join(fake.calls[0])
        assert "indexing.wasb.decode_backend=cpu" in joined
        assert "indexing.wasb.batch_size" not in joined
        assert "indexing.wasb.prefetch_batches" not in joined


    def test_cloud_failure_marks_failed_no_rows(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
        db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
        fake = _FakeCloudClient(returncode=1)
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []
        # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
        assert (
            out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
        )


    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path


    def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
        """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
```

```
input_root it
275         is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
276         The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
277         db, _bucket, _mp = cloud_env
278         m.global_config.storage.input_backend = "gcs"
279         m.global_config.storage.input_root = "gs://in-bucket/videos"
280         puts: list = []
281
282         def _fake_put(src, uri, **kw):
283             puts.append((str(src), uri, kw.get("config")))
284             return uri
285
286         monkeypatch.setattr(m.storage, "put", _fake_put)
287         fake = _FakeCloudClient()
288         _inject_fake(monkeypatch, fake)
289         local_path = str(tmp_path / "upload.mp4")
290
291         out = m._execute_processing(
292             m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
293         )
294
295         assert out["status"] == "success"
296         # 1) the local upload was staged UP to the input bucket, namespaced by video_id
297         assert len(puts) == 1
298         src, dest, put_cfg = puts[0]
299         assert src == local_path
300         assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
301         # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
302         # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
303         assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
304         assert put_cfg.get("input_backend") == "gcs"
305         # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
306         assert len(fake.calls) == 1
307         argv = fake.calls[0]
308         assert argv[argv.index("--video-uri") + 1] == dest
309         # 3) the DB video record now points at the staged source (so compile re-fetches it)
310         row = db.get_video(out["video_id"])
311         assert row is not None and row["filepath"] == dest
312
313
314     def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
315         cloud_env, tmp_path, monkeypatch
316     ):
317         """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid
cloud config ?
318         the storage layer classifies an explicit root by its scheme (resolve_input_uri joins
under
319         input_root before ever consulting input_backend), so the staging gate must too (PR
#467 review).
320         Regression: the gate previously required input_backend != 'local' and hard-failed
this shape."""
321         _db, _bucket, _mp = cloud_env
322         m.global_config.storage.input_root = "gs://in-bucket/videos"
323         # input_backend deliberately NOT set ? stays at the model default 'local'
324         assert m.global_config.storage.input_backend == "local"
325         puts: list = []
326
327         def _fake_put(src, uri, **kw):
328             puts.append((str(src), uri))
329             return uri
330
```

```
        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"  # NOT the 'needs a cloud input' pre-dispatch
failure
        assert len(puts) == 1
        src, dest = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # dispatch continued from the STAGED gs:// URI
        assert len(fake.calls) == 1
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == dest


    def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
        """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-
fail (no
        dispatch), rather than dispatching a path the Cloud Run job can't read."""
        _db, _bucket, _mp = cloud_env  # fixture leaves input_root unset
(input_backend=local)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        called = []
        monkeypatch.setattr(m.storage, "put", lambda *a, **k: called.append(a))

        out = m._execute_processing(
            m.ProcessRequest(video_path=str(tmp_path / "up.mp4"), segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == [] and called == []  # neither staged nor dispatched


    def test_default_off_runs_in_process(tmp_path, monkeypatch):
        """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
        byte-identically (the gate is inert). No Cloud Run dispatch."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig()  # compute_target defaults to ""  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        class _InProcSeg:
            def __init__(self, db_, cfg_):
                self.db = db_

            def process_video(self, vid, path, *a, **k):
                sid = self.db.add_play_segment(vid, 0.0, 5.0)
                return [
```

```
                    {
                        "id": sid,
                        "start_time": 0.0,
                        "end_time": 5.0,
                        "duration": 5.0,
                        "metadata": {},
                    }
                ]

        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        out = m._execute_processing(
            m.ProcessRequest(video_path=str(video), segmenter="motion")
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 1
        assert "cloud_run" not in out["message"]


    def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
        """A rc=0 marker but a MISSING intervals.csv (a partial push) must fail cleanly +
leak no rows
        (destroy-AFTER-confirm), not 500 or hang."""
        db, _bucket, monkeypatch = cloud_env

        class _NoCsvClient(_FakeCloudClient):
            def run_job(self, job_name, argv, *, region, project=None):
                self.calls.append(list(argv))
                out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
                done_uri = argv[argv.index("--done-uri") + 1]
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "status": "success"}, f)
                # deliberately DO NOT write intervals.csv
                os.makedirs(os.path.dirname(done_uri), exist_ok=True)
                with open(done_uri, "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "returncode": 0}, f)
                return "exec"

        _inject_fake(monkeypatch, _NoCsvClient())
        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "could not read the result" in out["message"]
        assert db.query_play_segments(out["video_id"], {}) == []


    def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
        """If the worker re-hashes different bytes (result.video_id != local id), ingest
still proceeds
        under the LOCAL id but logs a WARNING so the orphaned-report condition is
diagnosable."""
        db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
            )
        assert out["status"] == "success"
        assert len(db.query_play_segments(out["video_id"], {})) == 2
        assert any(
            "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
            for r in caplog.records
        )
```

```
453
454
455     class _InProcSeg:
456         """Minimal in-process segmenter (writes one play row) ? proves the local path
actually ran."""
457
458         def __init__(self, db_, cfg_):
459             self.db = db_
460
461         def process_video(self, vid, path, *a, **k):
462             sid = self.db.add_play_segment(vid, 0.0, 5.0)
463             return [
464                 {
465                     "id": sid,
466                     "start_time": 0.0,
467                     "end_time": 5.0,
468                     "duration": 5.0,
469                     "metadata": {},
470                 }
471             ]
472
473
474     # --- item 3: SPA compute-picker (per-request compute override + cloud_available
capability) ---
475
476
477     def test_per_request_local_forces_in_process(cloud_env):
478         """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
479         forces the in-process segmenter and NEVER dispatches to Cloud Run."""
480         db, _bucket, monkeypatch = cloud_env
481         fake = _FakeCloudClient()
482         _inject_fake(monkeypatch, fake)
483         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
484
485         out = m._execute_processing(
486             m.global_config,
487             m.db_instance,
488             m.ProcessRequest(
489                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
490             ),
491         )
492         assert out["status"] == "success"
493         assert len(out["play_segments"]) == 1
494         assert "cloud_run" not in out["message"]
495         assert (
496             fake.calls == []
497         )  # explicit "run on my machine" ? no dispatch despite a cloud server
498
499
500     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
501         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
forces a Cloud
502         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
a bucket)."""
503         db = Database(str(tmp_path / "t.db"))  # noqa: F841
504         monkeypatch.setattr(m, "db_instance", db)
505         bucket = tmp_path / "bucket"
506         bucket.mkdir()
507         cfg = AppConfig.model_validate(
508             {  # noqa: F841
509                 "deployment": {"compute_target": ""},  # server default = in-process
510                 "storage": {"backend": "local", "output_root": str(bucket)},
511             }
512         )
```

```
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
        monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
            ),
        )
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        assert (
            len(fake.calls) == 1
        )  # per-request override dispatched despite the LOCAL server default


    def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
        """compute='cloud' on a server NOT wired for it fails clearly (no silent local
fallback, no
        dispatch) so the picker can surface 'cloud isn't available here'."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        cfg = AppConfig.model_validate(
            {"deployment": {"compute_target": ""}}
        )  # no bucket, no env  # noqa: F841
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
            ),
        )
        assert out["status"] == "failed"
        assert "isn't configured" in out["message"]


    def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
        """An unrecognised compute value is ignored (logged) and the server default applies
? here a
        cloud_run server still dispatches to the cloud."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
```

```
        with caplog.at_level("WARNING", logger="backend.main"):
            out = m._execute_processing(
                m.global_config,
                m.db_instance,
                m.ProcessRequest(
                    video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
                ),
            )
        assert out["status"] == "success"
        assert len(fake.calls) == 1  # fell back to the configured cloud_run default
        assert any("unknown compute" in r.getMessage() for r in caplog.records)


    def test_capabilities_reports_cloud_available(cloud_env):
        """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
        cloud toggle only then)."""
        _db, _bucket, _monkeypatch = cloud_env
        app = m.create_app(m.global_config, m.db_instance)
        client = TestClient(app)
        resp = client.get("/api/capabilities")
        caps = resp.json()
        assert caps["deployment"]["cloud_available"] is True
        assert caps["deployment"]["compute_target"] == "cloud_run"


    def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
        """A default (local) server with no Cloud Run env advertises
cloud_available=False."""
        cfg = AppConfig()
        db = Database(str(tmp_path / "t.db"))
        app = m.create_app(cfg, db)
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        client = TestClient(app)
        resp = client.get("/api/capabilities")
        caps = resp.json()
        assert caps["deployment"]["cloud_available"] is False


    def test_full_api_process_to_query_via_cloud(cloud_env):
        """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
        to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient(video_id="vidCloud2")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid"},
        )
        assert r.status_code == 202
        job_id = r.json()["job_id"]

        job = client.get(f"/api/jobs/{job_id}").json()
        assert job["status"] == "done"
        video_id = job["video_id"]
        assert video_id

        q = client.post("/api/query", json={"video_id": video_id})
        assert q.status_code == 200
        segs = q.json()["play_segments"]
```

```
        assert len(segs) == 2
        assert all(_meta(s)["source"] == "cloud_run" for s in segs)


    def test_local_override_survives_api_boundary(cloud_env):
        """REGRESSION: the picker choice must survive the async submit?worker?reconstruct
boundary. On a
        cloud_run server, POST /api/process with compute='local' must persist (jobs.compute)
+ reconstruct
        (_job_to_request) the choice and run IN-PROCESS ? not dispatch to Cloud Run. (An
earlier cut
        dropped compute in create_job/_job_to_request, silently nullifying the picker
through /api/process.)"""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "motion",
                "compute": "local",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            fake.calls == []
        )  # 'local' honoured across the boundary ? never dispatched to Cloud Run
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 1
        assert all(
            _meta(s).get("source") != "cloud_run" for s in segs
        )  # in-process rows, not cloud


    def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
        """REGRESSION (mirror): on a LOCAL-default server wired for cloud (env + bucket),
POST /api/process
        with compute='cloud' must survive the boundary and dispatch to Cloud Run."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": ""},  # server default = in-process
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
```

```
        monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
        monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
        fake = _FakeCloudClient(video_id="vidBoundary")
        _inject_fake(monkeypatch, fake)
        app = m.create_app(m.global_config, db)
        client = TestClient(app)

        r = client.post(
            "/api/process",
            json={
                "video_path": "gs://b/clip.mp4",
                "segmenter": "wasb_hybrid",
                "compute": "cloud",
            },
        )
        assert r.status_code == 202
        job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
        assert job["status"] == "done"
        assert (
            len(fake.calls) == 1
        )  # 'cloud' honoured across the boundary ? dispatched despite local default
        segs = db.query_play_segments(job["video_id"], {})
        assert len(segs) == 2
        assert all(_meta(s)["source"] == "cloud_run" for s in segs)


    # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-------------
    # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
    # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
    # FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
    # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).


    def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
        """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
        a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
        _db, _bucket, _mp = cloud_env
        monkeypatch.delenv("WASB_WEIGHTS", raising=False)
        m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
            )
        )
        assert out["status"] == "failed"
        assert "can't run detection locally" in out["message"]
        assert "Run in cloud" in out["message"]
        assert "weights not found" not in out["message"]  # NOT the cryptic deep failure
        assert fake.calls == []  # never dispatched
        # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
        assert out["compute"]["requested"] == "local"
        assert out["compute"]["dispatched"] is False
        assert out["compute"]["path"] == "not_run"

```

```
755
756     def test_local_on_cloud_server_with_weights_present_runs_in_process(
757         cloud_env, tmp_path, monkeypatch
758     ):
759         """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
760         detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
761         _db, _bucket, _mp = cloud_env
762         weights = tmp_path / "wasb_badminton_best.pth.tar"
763         weights.write_bytes(b"WEIGHTS")
764         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
765         m.global_config.indexing.wasb.weights_path = str(weights)
766         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
767         fake = _FakeCloudClient()
768         _inject_fake(monkeypatch, fake)
769
770         out = m._execute_processing(
771             m.ProcessRequest(
772                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
773             )
774         )
775         assert out["status"] == "success"
776         assert fake.calls == []  # in-process, not dispatched
777         assert out["compute"]["path"] == "in_process"
778
779
780     def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
781         cloud_env, monkeypatch
782     ):
783         """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
784         compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
785         _db, _bucket, _mp = cloud_env
786         m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
787         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
788         fake = _FakeCloudClient()
789         _inject_fake(monkeypatch, fake)
790
791         out = m._execute_processing(
792             m.ProcessRequest(
793                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
794             )
795         )
796         assert out["status"] == "success"
797         assert fake.calls == []
798         assert out["compute"]["path"] == "in_process"
799
800
801     def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
802         tmp_path, monkeypatch
803     ):
804         """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
805         weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
806         'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
807         db = Database(str(tmp_path / "t.db"))  # noqa: F841
808         monkeypatch.setattr(m, "db_instance", db)
809         cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}})  # noqa: F841
810         monkeypatch.setattr(m, "global_config", cfg)
811         monkeypatch.setattr(
812             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
```

```
        )
        monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
        monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
        monkeypatch.delenv("WASB_WEIGHTS", raising=False)
        cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        out = m._execute_processing(
            m.ProcessRequest(
                video_path=str(video), segmenter="wasb_hybrid", compute="local"
            )
        )
        # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
        assert out["status"] == "success"
        assert out["compute"]["path"] == "in_process"


    # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----------------


    def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
        """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
        (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
        provable from the job result alone (no guessing from segment metadata)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
            )
        )
        prov = out["compute"]
        assert prov["path"] == "cloud_run"
        assert prov["requested"] == "cloud"
        assert (
            prov["execution"] == "exec-1"
        )  # the fake client's execution name (real one in prod)
        assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
        assert prov["job"] and prov["region"]  # the dispatch coordinates are recorded


    def test_local_override_stamps_in_process_provenance(cloud_env):
        """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
        requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
        _db, _bucket, monkeypatch = cloud_env
        _inject_fake(monkeypatch, _FakeCloudClient())
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

        out = m._execute_processing(
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
            )
        )
        assert out["status"] == "success"
        assert out["compute"] == {"path": "in_process", "requested": "local"}

```

```
872
873     def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
874         """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-
identical local
875         path is now self-describing."""
876         db = Database(str(tmp_path / "t.db"))  # noqa: F841
877         monkeypatch.setattr(m, "db_instance", db)
878         monkeypatch.setattr(m, "db_instance", db)
879         monkeypatch.setattr(m, "global_config", AppConfig())
880         m.app.state.config = AppConfig()
881         monkeypatch.setattr(
882             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
883         )
884         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
885         video = tmp_path / "v.mp4"
886         video.write_bytes(b"hashable-bytes")
887
888         out = m._execute_processing(
889             m.ProcessRequest(video_path=str(video), segmenter="motion")
890         )
891         assert out["compute"] == {"path": "in_process", "requested": None}
892
893
894     def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
895         """compute='cloud' on a server that can't dispatch fails clearly AND records it:
target cloud_run,
896         requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
897         db = Database(str(tmp_path / "t.db"))  # noqa: F841
898         monkeypatch.setattr(m, "db_instance", db)
899         monkeypatch.setattr(m, "db_instance", db)
900         monkeypatch.setattr(
901             m,
902             "global_config",
903             AppConfig.model_validate({"deployment": {"compute_target": ""}}),
904         )
905         m.app.state.config = AppConfig.model_validate(
906             {"deployment": {"compute_target": ""}}
907         )
908         monkeypatch.setattr(
909             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
910         )
911         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
912         monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
913         video = tmp_path / "v.mp4"
914         video.write_bytes(b"hashable-bytes")
915
916         out = m._execute_processing(
917             m.ProcessRequest(
918                 video_path=str(video), segmenter="wasb_hybrid", compute="cloud"
919             )
920         )
921         assert out["status"] == "failed"
922         # Honest: it ran NOWHERE (pre-dispatch reject) ? path=not_run, but the intent is
recorded.
923         assert out["compute"] == {
924             "path": "not_run",
925             "requested": "cloud",
926             "target": "cloud_run",
927             "dispatched": False,
928         }
929
930
931     def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
932         """A validation failure never reaches a segmenter ? path=not_run (nothing
```

```
executed)."""
933         db = Database(str(tmp_path / "t.db"))  # noqa: F841
934         monkeypatch.setattr(m, "db_instance", db)
935         monkeypatch.setattr(m, "global_config", AppConfig())
936         m.app.state.config = AppConfig()
937         fail = [
938             ValidationResult(validator_name="x", passed=False, message="bad", details={})
939         ]
940         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
941         video = tmp_path / "v.mp4"
942         video.write_bytes(b"hashable-bytes")
943 
944         out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
945         assert out["status"] == "failed"
946         assert out["compute"]["path"] == "not_run"
947 
948 
949     def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
950         """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls
carries the
951         cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was
taken."""
952         _db, _bucket, monkeypatch = cloud_env
953         _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
954         app = m.create_app(m.global_config, _db)
955         client = TestClient(app)
956 
957         r = client.post(
958             "/api/process",
959             json={
960                 "video_path": "gs://b/clip.mp4",
961                 "segmenter": "wasb_hybrid",
962                 "compute": "cloud",
963             },
964         )
965         assert r.status_code == 202
966         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
967         assert job["status"] == "done"
968         assert job["result"]["compute"]["path"] == "cloud_run"
969         assert job["result"]["compute"]["execution"] == "exec-1"
970 
971 
972     # --- #468: local compute lane ? CPU/GPU-heavy in-process work serializes, remote polls
don't --------
973 
974 
975     def test_in_process_detection_runs_under_the_local_lane(cloud_env):
976         """The in-process segmenter holds orchestration._LOCAL_COMPUTE_LANE while it runs,
so raising the
977         drain worker count (to overlap I/O-bound remote dispatch) never runs two local
detections at once.
978         A remote dispatch returns before this path, so remote polls never hold the lane
(they overlap)."""
979         import backend.orchestration as orch
980 
981         _db, _bucket, monkeypatch = cloud_env
982         held = {}
983 
984         class _LaneCheckSeg:
985             def __init__(self, db_, cfg_):
986                 self.db = db_
987 
988             def process_video(self, vid, path, *a, **k):
989                 # the wrapping `with _LOCAL_COMPUTE_LANE` holds the size-1 lane ? a non-
blocking re-acquire
```

```
990                 # must FAIL while detection runs (proving concurrent local detections are
excluded).
991                 got = orch._LOCAL_COMPUTE_LANE.acquire(blocking=False)
992                 held["busy"] = not got
993                 if got:
994                     orch._LOCAL_COMPUTE_LANE.release()
995                 sid = self.db.add_play_segment(vid, 0.0, 5.0)
996                 return [
997                     {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}
998                 ]
999
1000        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _LaneCheckSeg)
1001        out = m._execute_processing(
1002            m.global_config,
1003            m.db_instance,
1004            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion",
compute="local"),
1005        )
1006        assert out["status"] == "success"
1007        assert held.get("busy") is True  # detection ran while the local lane was held
1008
1009
1010    def test_wait_progress_message_is_honest_and_minute_granular():
1011        """#482: the CP knows ELAPSED time, not frames ? the heartbeat says so plainly,
shows
1012        movement from the first tick ("starting up" during queue/cold-start), then counts
minutes."""
1013        from backend.orchestration import _wait_progress_message
1014
1015        early = _wait_progress_message("wasb_hybrid", 12.0)
1016        assert "wasb_hybrid" in early and "starting up" in early
1017        later = _wait_progress_message("wasb_hybrid", 125.0)
1018        assert later.endswith("2 min elapsed")
1019
```

11. assistant (2026-07-07T15:17:54.030Z)

I have both authoritative files. Let me verify the finding's claims against the actual code by tracing the specific scenario.

Claim verification:

1. **Guardrail requires `_cloud_available(cfg)` True** ? orchestration.py:488-497. The `choice == "local"` fast-fail requires `seg_name in _WEIGHTS_BACKED_SEGMENTERS and _cloud_available(cfg) and not _local_detector_weights_present(...)`. Confirmed.

2. **`_cloud_available` has two routes** ? orchestration.py:199-206: config route (`compute_target == "cloud_run"` ? True) and env route (`CLOUD_RUN_JOB and GOOGLE_CLOUD_PROJECT and storage.output_root`). Confirmed.

3. **Only guardrail-FIRES test uses config route** ? Scanning every compute="local" + weights-backed test: the *only* test asserting the fast-fail is `test_local_on_cloud_server_without_weights_fails_fast` (731-753), which uses the `cloud_env` fixture (`compute_target=cloud_run`, config route). The "not guardrailed" test (801-829) sets `compute_target=''` AND deletes both env vars. So no test drives the guardrail firing via the env route. The coverage gap is factually real.

But is the described *failure* real against the code as written? Let me trace the env-wired scenario concretely: `compute_target=''`, `CLOUD_RUN_JOB='rally-worker'`, `GOOGLE_CLOUD_PROJECT='proj'`, `output_root='gs://bucket'`, `compute='local'`, `segmenter='wasb_hybrid'`, no local weights:

- `choice == "local"` ? enter block (line 480)
- `'wasb_hybrid' in _WEIGHTS_BACKED_SEGMENTERS` ? True

- `_cloud_available(cfg)`: compute_target='' skips config return; `has_job = bool('rally-worker' and 'proj')` = True; `out_root='gs://bucket'` truthy ? returns **True** (env route)
- `not _local_detector_weights_present(...)`: path='/nonexistent', `os.path.exists` False ? returns False ? `not False` = True
- All three True ? **guardrail FIRES**, `_failed("...can't run detection locally...", False)`

The code fires the guardrail correctly via the env route. The described failure ("silently fall through to the deep 'weights not found'") does **not** occur against the real code.

Let me confirm the env route of `_cloud_available` is regression-protected by existing tests, and check `main.py`/config for completeness.

12. user (2026-07-07T15:17:56.532Z)

<tool_use_error>InputValidationError: Grep failed due to the following issue:
An unexpected parameter `command` was provided</tool_use_error>

13. user (2026-07-07T15:17:57.279Z)

```
360             "ui": _bundled_ui_version(),
361         }
362
363
364     @app.get("/api/capabilities")
365     def capabilities(request: Request):
366         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
367         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
368         value. Best-effort + never raises (the wizard degrades gracefully)."""
369         cfg = request.app.state.config
370         indexing = getattr(cfg, "indexing", None)
371         deployment = getattr(cfg, "deployment", None)
372         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
373         ffprobe = shutil.which(ffprobe_bin())
374         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
375         wasb = getattr(indexing, "wasb", None)
376         weights_path = (
377             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
378         )
379         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
380         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
381         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
382         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
383         nvidia_smi = shutil.which("nvidia-smi")
384         return {
385             "api_version": API_VERSION,
386             "engine_version": _engine_version(),
387             "segmenters": sorted(segmenter_registry.list_available().keys()),
388             "default_segmenter": getattr(indexing, "default_segmenter", None),
389             "ffmpeg": {
390                 "ffmpeg": bool(ffmpeg),
391                 "ffprobe": bool(ffprobe),
392                 "ffmpeg_path": ffmpeg,
393                 "ffprobe_path": ffprobe,
394             },
395             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
396             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
397             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
398             **uploads_api.upload_caps(cfg),  # #480 item 5: the SPA pre-flight reads
max_upload_mb
```

```
399             "deployment": {
400                 "profile": getattr(deployment, "profile", "") or "",
401                 "compute_target": getattr(deployment, "compute_target", "") or "",
402                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
403                 # the "run in the cloud" toggle only when true (else it's local-only).
404                 "cloud_available": _cloud_available(cfg),
405             },
406             # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
407             # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
408             "storage": storage_capabilities(
409                 getattr(cfg, "storage", None), local_free_path=output_base(cfg)
410             ),
411         }
412
413
414     def _friendly_video_name(filepath: str) -> str:
415         """A human name for a video from its stored path/URI ? the filename stem
(``?/match_2026.mp4`` ?
416         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
417         Falls back to the whole ``filepath`` if there's no basename."""
418         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
419         stem = os.path.splitext(base)[0]
```

14. assistant (2026-07-07T15:18:01.876Z)

Let me confirm the env-route regression protection: which tests would fail if `_cloud_available`'s env route regressed.

15. user (2026-07-07T15:18:04.990Z)

```
502:    Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT + a
bucket)."""
523:    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
554:    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
567:    assert "isn't configured" in out["message"]
589:def test_capabilities_reports_cloud_available(cloud_env):
606:    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
698:    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
814:    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
911:    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
```

16. assistant (2026-07-07T15:18:31.301Z)

I now have everything needed to reach a verdict. Let me record the findings.

Verification complete. Key facts established against the authoritative code:

- **The coverage gap is factually real**: the only guardrail-FIRES test is `test_local_on_cloud_server_without_weights_fails_fast` (uses `cloud_env` ? config route `compute_target=cloud_run`). No test drives the guardrail firing via the env route on a `compute_target=''` box.

- **But the described failure does NOT occur against the real code.** I traced the env-wired scenario (`compute_target=''` + `CLOUD_RUN_JOB` + `GOOGLE_CLOUD_PROJECT` + `output_root` + `compute='local'` + `wasb_hybrid` + no weights) through `_maybe_dispatch_remote`: `_cloud_available` returns True via the env route (orchestration.py:202-206), and the guardrail (488-497) **fires correctly** ? fast-fail "can't run detection locally", never dispatches, `path=not_run`. The code is correct; there is no wrong behavior in the change under review.

- **The finding assumes a code path that doesn't exist.** There is no separate "env branch of

the guardrail's fire condition" ? the guardrail calls `_cloud_available(cfg)` as a single uniform boolean. The config-vs-env distinction lives entirely inside `_cloud_available`; it creates no distinct branch within the guardrail.

- **The env route is already regression-protected.** Confirmed via grep: `test_per_request_cloud_forces_dispatch_on_local_server` (setenv CLOUD_RUN_JOB, line 523) and `test_cloud_override_survives_api_boundary_on_local_server` (line 698) both run `compute='cloud'` on a `compute_target=''` box and require dispatch ? so if `_cloud_available`'s env route regressed to False, both would fail with "isn't configured". The guardrail consumes that same tested function.

The finding's failure is explicitly conditional ("*if* that env branch... regressed") and self-concedes "real-world risk is low ? a completeness gap, not a likely latent bug." It does not meet the review's bar of a concrete config ? wrong result against the code as written.

17. user (2026-07-07T15:18:45.650Z)

Structured output provided successfully